

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06

NAME: A SHIVA SHANKAR REDDY

REG NO:192425174 CO 4:

COURSE OUTCOME COVERED

CO5: Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving. (BL5,BL6)

1. Problem Statement

The N-Queens Problem requires placing N chess queens on an $N \times N$ chessboard such that no two queens attack each other. Two queens attack each other if they share the same row, the same column, or the same diagonal. The task is to design a well-structured pseudocode using the Backtracking technique that systematically places queens one row at a time, checks the safety of each placement, and backtracks (undoes the last placement) whenever no safe column is available in the current row, until either a valid full placement is found or all possibilities have been exhausted.

1.1 Input

- N – a positive integer representing both the size of the chessboard ($N \times N$) and the number of queens to be placed.

1.2 Expected Output

- One or more valid board configurations (queen placements) such that no two queens attack each other, typically represented as an array 'board[0..N-1]', where board[row] gives the column index of the queen placed in that row.
- If no valid configuration exists for the given N (this only happens for $N = 2$ and $N = 3$), the algorithm should report that no solution exists.
- Optionally, the total count of distinct valid solutions for the given N .

2. Backtracking Approach – Design Rationale

Backtracking is well suited to the N-Queens problem because it incrementally builds a solution (placing one queen per row) and abandons ('backtracks from') a partial placement as soon as it is discovered to be invalid, rather than generating and checking every possible arrangement of N queens on $N \times N$ squares. Since exactly one queen must be placed in each row (two queens can never share a row in a valid solution), the search space is reduced to choosing one column per row, and the algorithm only needs to verify column and diagonal safety against queens already placed in previous rows.

The key design decisions are: (1) process the board row by row, attempting to place a queen in each column of the current row; (2) before placing, check whether the position is 'safe' – i.e., not attacked by any previously placed queen via row, column, or either diagonal; (3) if safe, place the queen and recursively attempt to solve the next row; (4) if the recursive call fails to complete a valid placement, remove ('backtrack') the queen and try the next column; (5) if all columns in a row are tried without success, return failure to the previous row, which will then try its next column.

3. Safety Conditions for Queen Placement

A queen placed at (row, col) is considered SAFE with respect to an already-placed queen at (r, c) if all of the following hold simultaneously:

Condition	Rule	Explanation
Row conflict	$\text{row} \neq r$	Automatically satisfied since exactly one queen is placed per row by construction
Column conflict	$\text{col} \neq c$	No other queen already occupies the same column
Diagonal conflict (/)	$\text{row} - \text{col} \neq r - c$	No queen shares the same '/' diagonal
Diagonal conflict (\)	$\text{row} + \text{col} \neq r + c$	No queen shares the same '\' diagonal

In an efficient implementation, three boolean sets/arrays – cols[], diag1[] (indexed by row – col), and diag2[] (indexed by row + col) – are maintained so that safety can be checked in O(1) time per candidate column instead of re-scanning all previously placed queens.

4. Pseudocode (Backtracking)

The following pseudocode places queens row by row and prunes invalid branches immediately using the safety conditions above:

```
ALGORITHM SolveNQueens(N)
    INPUT : N                                // size of the chessboard / number of
    queens
    OUTPUT: All valid board configurations (or 'No solution exists')

    board[0 .. N-1] <- -1                    // board[row] = column of queen in that row
    cols[0 .. N-1] <- FALSE                  // cols[c]      = TRUE if column c is
    occupied
    diag1[0 .. 2N-2] <- FALSE                // diag1[row-col+N-1] = TRUE if that '/'
    diagonal is occupied
    diag2[0 .. 2N-2] <- FALSE                // diag2[row+col]      = TRUE if that '\'
    diagonal is occupied
    solutionCount <- 0

    CALL PlaceQueen(0, N)

    IF solutionCount = 0 THEN
        PRINT 'No solution exists for N =', N
    ELSE
        PRINT solutionCount, 'solution(s) found'
    END IF

PROCEDURE PlaceQueen(row, N)
    IF row = N THEN                          // base case: all N queens placed
    safely
        solutionCount <- solutionCount + 1
        CALL PrintBoard(board, N)           // record / display this valid
    configuration
        RETURN
    END IF
```

```

    FOR col <- 0 TO N-1 DO                                // try every column in the current
row
    IF IsSafe(row, col, N) THEN
        // ---- place the queen ----
        board[row] <- col
        cols[col] <- TRUE
        diag1[row - col + N - 1] <- TRUE
        diag2[row + col] <- TRUE

        CALL PlaceQueen(row + 1, N)                      // recurse to the next row

        // ---- backtrack: undo the placement ----
        board[row] <- -1
        cols[col] <- FALSE
        diag1[row - col + N - 1] <- FALSE
        diag2[row + col] <- FALSE
    END IF
    // if not safe, this column is pruned automatically (no recursive call
made)
END FOR
END PROCEDURE

FUNCTION IsSafe(row, col, N) RETURNS BOOLEAN
    IF cols[col] = TRUE THEN
        RETURN FALSE                                     // column already attacked
    END IF
    IF diag1[row - col + N - 1] = TRUE THEN
        RETURN FALSE                                     // '/' diagonal already attacked
    END IF
    IF diag2[row + col] = TRUE THEN
        RETURN FALSE                                     // '\' diagonal already attacked
    END IF
    RETURN TRUE                                           // position is safe
END FUNCTION

```

5. Logical Correctness – Why No Two Queens Attack Each Other

- Row safety is guaranteed structurally: the algorithm places at most one queen per row (the recursion advances row by row and never revisits a row), so two queens can never share a row.
- Column safety is guaranteed by the cols[] check inside IsSafe(): a column already marked TRUE is never reused, so no two queens can share a column.
- Diagonal safety (both directions) is guaranteed by the diag1[] and diag2[] checks: any cell lying on a diagonal already occupied by a placed queen is rejected before placement.
- Because IsSafe() is evaluated before every placement, and the corresponding cols[]/diag1[]/diag2[] flags are correctly set on placement and cleared on backtracking, every partial and complete board maintained by the algorithm at any point in the recursion is guaranteed to be a valid, non-attacking configuration.

6. Pruning of Invalid Placements

The algorithm prunes the search space in two complementary ways. First, structurally: by placing exactly one queen per row, the search space is reduced from choosing N positions among N^2 cells (an enormous combinatorial space) to choosing one column out of N for each of the N rows. Second, dynamically: the `IsSafe()` check is performed before recursing into the next row, so an unsafe column is rejected immediately – the algorithm never explores any deeper branch of the recursion tree that starts from an invalid partial placement. This is the defining characteristic of backtracking with pruning: invalid partial solutions are eliminated as early as possible ('fail fast'), rather than being completed and then checked for validity, which is what a brute-force approach would do.

7. Worked Example ($N = 4$)

For a 4×4 board, the algorithm finds exactly 2 valid solutions. One such solution, expressed as `board[row] = column`, is `board = [1, 3, 0, 2]`, meaning: queen in row 0 is placed at column 1, row 1 at column 3, row 2 at column 0, and row 3 at column 2. The second solution is the mirror image, `board = [2, 0, 3, 1]`. Both configurations satisfy all row, column, and diagonal safety conditions from Section 3.

N (Board Size)	Number of Valid Solutions
4	2
8	92

8. Implementation (Python) – for Verification of the Pseudocode

The pseudocode in Section 4 was implemented in Python to verify its correctness against known results (2 solutions for $N=4$, 92 solutions for $N=8$):

```
def solve_n_queens(n):
    solutions = []
    board = [-1] * n
    cols, diag1, diag2 = set(), set(), set()

    def is_safe(row, col):
        return col not in cols and (row - col) not in diag1 and (row + col)
        not in diag2

    def place_queen(row):
        if row == n:
            solutions.append(board.copy())          # base case: full valid
        placement
        return
        for col in range(n):                        # try every column in this
        row
            if is_safe(row, col):
                board[row] = col                    # place
                cols.add(col)
                diag1.add(row - col)
                diag2.add(row + col)

                place_queen(row + 1)                # recurse to next row
```

```

        cols.remove(col)                    # backtrack: undo placement
        diag1.remove(row - col)
        diag2.remove(row + col)
    # unsafe columns are skipped (pruned) with no recursive call

    place_queen(0)
    return solutions

if __name__ == '__main__':
    for n in (4, 8):
        result = solve_n_queens(n)
        print(f'N = {n} -> {len(result)} solution(s)')
        print('Sample N=4 solutions:', solve_n_queens(4))

```

8.1 Program Output

```

N = 4 -> 2 solution(s)
N = 8 -> 92 solution(s)
Sample N=4 solutions: [[1, 3, 0, 2], [2, 0, 3, 1]]

```

This output matches the known, mathematically established solution counts for the N-Queens problem, confirming that the pseudocode design in Section 4 is logically correct.

Ouput:

The screenshot shows a Python Online Compiler interface. On the left, a sidebar contains icons for file management and execution. The main editor displays the code for the N-Queens problem, with line numbers 13 to 35. The code defines functions for solving the problem, including a recursive function to place queens row by row. On the right, the 'Output' panel shows the results of running the program, which match the expected output: 2 solutions for N=4 and 92 solutions for N=8, along with sample solutions for N=4 and a visual representation of the first solution.

```

main.py Visualize Run
13 def solve_n_queens(n):
14     solutions = []
15     board = [-1] * n          # board[row] = column of queen in
                                # that row
16     cols, diag1, diag2 = set(), set(), set()
17
18     def is_safe(row, col):
19         return (col not in cols and
20                 (row - col) not in diag1 and
21                 (row + col) not in diag2)
22
23     def place_queen(row):
24         if row == n:
25             solutions.append(board.copy()) # base case: full valid
26             placement
27             return
28         for col in range(n):              # try every column in
29             if is_safe(row, col):          # this row
30                 # ---- place the queen ----
31                 board[row] = col
32                 cols.add(col)
33                 diag1.add(row - col)
34                 diag2.add(row + col)
35                 place_queen(row + 1)      # recurse to next row

```

Output

```

N = 4 -> 2 solution(s)
N = 8 -> 92 solution(s)

Sample N=4 solutions: [[1, 3, 0, 2], [2, 0, 3, 1]]

First N=4 board layout:
. Q . .
. . . Q
Q . . .
. . Q .

=== Code Execution Successful ===

```

9. Suggested Tools and Platforms for Implementation

- Programming language: Python 3 (or Java/C++) to implement and test the backtracking algorithm described in the pseudocode.
- IDE / Editor: Visual Studio Code or PyCharm for development, debugging, and stepping through recursive calls.
- Version control & submission: Git and GitHub, for hosting the Source_Code and Report as required by the submission instructions.
- Visualization (optional): A simple console or Matplotlib-based chessboard renderer to visually display each valid queen placement found.
- Testing: Manual verification against known solution counts ($N=4 \rightarrow 2$, $N=6 \rightarrow 4$, $N=8 \rightarrow 92$) to validate correctness.

10. Performance Analysis – Time and Space Complexity

Approach	Time Complexity	Space Complexity
Brute Force (check all N^2 choose N placements)	Exponential / factorial – far worse than $O(N^N)$	$O(N)$
Backtracking (row-by-row with pruning)	$O(N!)$ worst case – significantly pruned in practice via <code>IsSafe()</code>	$O(N)$ for <code>board[]</code> , <code>cols[]</code> , <code>diag1[]</code> , <code>diag2[]</code> + $O(N)$ recursion depth

The unrestricted brute-force approach (trying all possible ways to place N queens anywhere on the board) is computationally infeasible even for moderate N . By restricting the search to one queen per row, the backtracking algorithm reduces the branching factor at each level to at most N choices, giving a loose worst-case bound of $O(N!)$. In practice, the `IsSafe()` pruning eliminates the vast majority of these branches long before a full row assignment is attempted, making the algorithm run efficiently for board sizes typically used in coursework and many real applications (N up to several dozen). Space usage is $O(N)$ for the board and the three tracking sets/arrays, plus $O(N)$ for the recursion call stack, since the recursion depth never exceeds N (one call per row).

11. Conclusion

The N -Queens problem is efficiently and correctly solved using a Backtracking algorithm that places one queen per row, verifies row, column, and both diagonal safety conditions in constant time via auxiliary tracking structures, and prunes any branch that leads to an unsafe partial configuration. The pseudocode presented in Section 4 is logically complete, uses well-structured loops and conditional checks, handles all board configurations (including reporting 'no solution' for $N = 2$ or $N = 3$), and was verified through a working Python implementation whose output (2 solutions for $N=4$, 92 solutions for $N=8$) matches the mathematically known results, confirming the correctness of the algorithm design.